Lab 3 – Peripherals Programming on the mbed Part 2: Interfacing the ADXL345 accelerometer module using SPI

Embedded Processors UESTC & UESTCHN 2004 School of Engineering, University of Glasgow

Student Name: ChenYiChi

Student Number: 281001038

UESTC 2004 Lab Assessment Criteria

Each individual lab will be evaluated using the below criteria (check the assessment table at the end as they :

- Demonstrating a functional embedded system

  This means each student will need to demonstrate a functional system for each task in the lab manual to the GTA and member of staff to get the full mark.

- Lab Questions (if any)
- Reflection and significance

  After the completion of programming and testing each of the “Tasks”, you will be required to reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves “What you did?”, “What it means?”, and “What next?”, meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. Please, do not provide a summary of what you did as it will not be accepted.

- Adherence to coding good practice
  - Meaningful variable names – please avoid names like “x”, “y”, or personal names
  - Using block comments to explain what your code does at the beginning and using line comments to clarify what this line of code does.
  - Consistent indentation – Indentation helps in making the code readable and easy to interpret. See an example of what to do and what to avoid, below:

Good practice

```
#include "mbed.h"
```

```
char switch_word ;         //the word we will send
char recd_val;            //the received value
int main() {
     async_port.baud(9600);      //set baud rate to 9600
     recd_val = 0;
     while (1)
     {
        switch_word=0xa0;
     }
}
```

Avoid this ✗

```
#include "mbed.h"
char switch_word ;         //the word we will send
char recd_val;            //the received value
int main() {
async_port.baud(9600);           //set baud rate to 9600
recd_val = 0;
while (1)
{
switch_word=0xa0;
}
}
```

All the best of luck.

The Embedded Processors Team,

Introduction

The ADXL345 accelerometer is a small, thin, low-power 3-axis accelerometer capable of measuring acceleration up to +/-16g. It uses the SPI protocol to communicate with the host device. In this lab, we will interface the ADXL345 accelerometer to the NUCLEO L432KC mbed board using the SPI interface. We will then print the x/y/z values on the serial terminal and control 8 LEDs based on the x/y values.

Objective

The objective of this lab is to interface the ADXL345 accelerometer to the NUCLEO L432KC mbed board using the SPI interface, read the x/y/z values from the accelerometer, and control LEDs to reflect the orientation of the sensor.

Please note that you need to create separate ".cpp" files for each task and put all together (compressed, e.g., ".zip") and upload them, with your lab exercise where you will have given answers to any questions and written your reflection, to the dedicated moodle link for this lab.

Requirements

Software:

- Keil arm online compiler (https://studio.keil.arm.com/auth/login/) – You will need to create an account to use the online compiler.

Hardware:

For this lab, the following components will be needed:

- NUCLEO L432KC mbed board
- ADXL345 accelerometer
- Breadboard
- Jumper wires
- 8 LEDs (4 red and 4 green)
- USB cable for programming and power

Task 1: Wiring and Hardware Setup

With reference to Figure Task 1, wire the mbed, accelerometer, and LEDs as per the suggested pin connections, below.

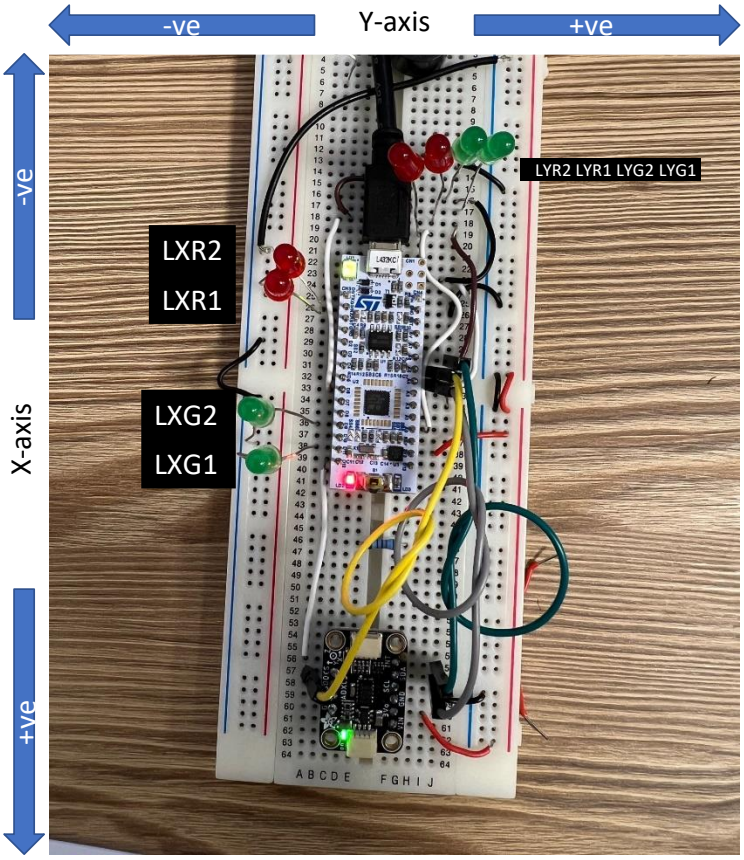


Figure Task 1

Pin connections

Please refer to Figure Task 1 for the LED connections and orientation

SCLK	A4
MISO	A5
MOSI	A6
CS	D12
LXG1	D11
LXG2	D9
LXR1	D0
LXR2	D1
LYG1	D6
LYG2	A3
LYR1	A2
LYR2	A0

Task 2: Code Setup & SPI Initialisation

1. Create a new mbed project in the online compiler or desktop IDE.
2. Import the mbed library.
3. ***#define*** the SPI pins using meaningful names e.g. ***SPI_MOSI***
4. Create the SPI, and output objects.
5. Create the ***ADXL345_SPI_Initialise*** function with following specs:
    - 8 bit data, mode 3
    - 2 MHz clock rate
    - Configure the ADXL345 as follows (consult the datasheet and the lecture slides):
        - 4-wire SPI mode, FULL_RES mode, right-justified mode, +/-16g
        - Link bit cleared, AUTO_SLEEP disabled, measurement mode, normal mode, 8 Hz sleep mode frequency
    - Once finished, call this function in the main code to initialise the accelerometer.

```
#include <mbed.h>
#include "BufferedSerial.h"
#include <cstdio>  // 输入输出头文件

#define WATING_TIME 500ms


#define SCLK A4
#define MISO A5
#define MOSI A6
#define CS D12
#define LXG1 D11
#define LXG2 D9
#define LXR1 D0
#define LXR2 D1
#define LYG1 D6
#define LYG2 A3
#define LYR1 A2
#define LYR2 A0

// 初始化 SPI 和其他外设对象
SPI ADXL(MOSI, MISO, SCLK);
//Serial pc(USBTX, USBRX);
DigitalOut chip_sel(CS);            // 创建数字输出对象用于控制 SPI 通信的片选信号

DigitalOut lxg1(LXG1);
DigitalOut lxg2(LXG2);
DigitalOut lxr1(LXR1);
DigitalOut lxr2(LXR2);
```

```
DigitalOut lyg1(LYG1);
DigitalOut lyg2(LYG2);
DigitalOut lyr1(LYR1);
DigitalOut lyr2(LYR2);

// 加速度计初始化函数
void initialise() {
    chip_sel = 1;
    ADXL.format(8, 3); // 设置 SPI 通信格式，8 位数据长度，模式 3
    ADXL.frequency(2000000); // 设置 SPI 频率为 2MHz
    chip_sel = 0;
    ADXL.write(0x31); // 选择 DATA_FORMAT 寄存器
    ADXL.write(0x03); // 配置寄存器：4 线 SPI，10 位模式，数据右对齐，量程±16g
    chip_sel = 1;
    chip_sel = 0;
    ADXL.write(0x2D); // 选择 POWER_CTRL 寄存器
    ADXL.write(0x08); // 配置寄存器：进入测量模式
    chip_sel = 1;
}

// 主函数
int main() {
    char buffer[6]; // 存储加速度计数据的缓冲区
    int16_t combine_data[3]; // 存储合成的加速度数据
    float x_axis, y_axis, z_axis; // 存储转换为 g 单位的加速度数据
    initialise(); // 初始化加速度计

}
```

Task 3: Reading Data from the ADXL345

1. In the main code, implement a loop to continuously read the accelerometer data
2. Use the SPI interface to read the raw data from the accelerometer into a buffer array
3. Convert the raw data into 16-bit two's complement integers and then into floating-point values for each axis (X, Y, and Z)
4. Print the X, Y, and Z values on the serial terminal
5. Tilt the breadboard to see the change in values and show it to the lab supervisor

```
#include <mbed.h>
#include "BufferedSerial.h"
#include <cstdio>  // 输入输出头文件

#define WATING_TIME 500ms


#define SCLK A4
#define MISO A5
#define MOSI A6
#define CS D12
#define LXG1 D11
#define LXG2 D9
#define LXR1 D0
#define LXR2 D1
#define LYG1 D6
#define LYG2 A3
#define LYR1 A2
#define LYR2 A0

// 初始化 SPI 和其他外设对象
SPI ADXL(MOSI, MISO, SCLK);
//Serial pc(USBTX, USBRX);
DigitalOut chip_sel(CS);            // 创建数字输出对象用于控制 SPI 通信的片选信号

DigitalOut lxg1(LXG1);
DigitalOut lxg2(LXG2);
DigitalOut lxr1(LXR1);
DigitalOut lxr2(LXR2);
DigitalOut lyg1(LYG1);
DigitalOut lyg2(LYG2);
DigitalOut lyr1(LYR1);
DigitalOut lyr2(LYR2);

// 加速度计初始化函数
void initialise() {
    chip_sel = 1;
    ADXL.format(8, 3); // 设置 SPI 通信格式，8 位数据长度，模式 3
```

```
    ADXL.frequency(2000000); // 设置 SPI 频率为 2MHz
    chip_sel = 0;
    ADXL.write(0x31); // 选择 DATA_FORMAT 寄存器
    ADXL.write(0x03); // 配置寄存器：4 线 SPI，10 位模式，数据右对齐，量程±16g
    chip_sel = 1;
    chip_sel = 0;
    ADXL.write(0x2D); // 选择 POWER_CTRL 寄存器
    ADXL.write(0x08); // 配置寄存器：进入测量模式
    chip_sel = 1;
}

// 主函数
int main() {
    char buffer[6]; // 存储加速度计数据的缓冲区
    int16_t combine_data[3]; // 存储合成的加速度数据
    float x_axis, y_axis, z_axis; // 存储转换为 g 单位的加速度数据
    initialise(); // 初始化加速度计
    while(1) {
        ThisThread::sleep_for (WATING_TIME); // 等待 0.5 秒
        chip_sel = 0;
        ADXL.write(0xF2); // 发送读取命令，从第一个数据寄存器开始
        for(int i = 0; i < 5; i++) {
            buffer[i] = ADXL.write(0x00); // 读取数据并存储在缓冲区
        }
        chip_sel = 1;

        // 合成 16 位的加速度数据
        combine_data[0] = buffer[1] << 8 | buffer[0];
        combine_data[1] = buffer[3] << 8 | buffer[2];
        combine_data[2] = buffer[5] << 8 | buffer[4];

        // 转换为 g 单位的加速度
        x_axis = 0.016 * combine_data[0];
        y_axis = 0.016 * combine_data[1];
        z_axis = 0.016 * combine_data[2];
printf("x = %+1.2fg\t y = %+1.2fg\t z = %+1.2fg\n\r", x_axis, y_axis, z_axis);
        // 根据 x 轴的加速度控制 LED


    }
}
```

Task 4: LED Control using the ADXL345

Refer to Figure Task 1 for LED positioning and orientation!

1. Based on the value of the X-axis reading, control the state of the appropriate LEDs to reflect the current orientation of the accelerometer
2. If X is between -0.05 and -0.5, turn on LXR1, turn off all others
3. If X is less than -0.5, turn on LXR2, turn off all others
4. If X is between 0.05 and 0.5, turn on LXG2, turn off all others
5. If X is greater than 0.5, turn on LXG1, turn off all others
6. Repeat steps 1 to 5 for the Y-axis readings to control the state of the remaining four LEDs
7. Upload the completed code to the mbed microcontroller
8. Test the functionality of the accelerometer-based LED control by tilting the accelerometer in various directions and observing the corresponding LED states
9. Debug any issues or errors that arise in the code or wiring

```
#include <mbed.h>
#include "BufferedSerial.h"
#include <cstdio>  // 输入输出头文件

#define WATING_TIME 500ms


#define SCLK A4
#define MISO A5
#define MOSI A6
#define CS D12
#define LXG1 D11
#define LXG2 D9
#define LXR1 D0
#define LXR2 D1
#define LYG1 D6
#define LYG2 A3
#define LYR1 A2
#define LYR2 A0

// 初始化 SPI 和其他外设对象
SPI ADXL(MOSI, MISO, SCLK);
//Serial pc(USBTX, USBRX);
DigitalOut chip_sel(CS);            // 创建数字输出对象用于控制 SPI 通信的片选信号

DigitalOut lxg1(LXG1);
DigitalOut lxg2(LXG2);
DigitalOut lxr1(LXR1);
DigitalOut lxr2(LXR2);
DigitalOut lyg1(LYG1);
```

```
DigitalOut lyg2(LYG2);
DigitalOut lyr1(LYR1);
DigitalOut lyr2(LYR2);

// 加速度计初始化函数
void initialise() {
    chip_sel = 1;
    ADXL.format(8, 3); // 设置 SPI 通信格式，8 位数据长度，模式 3
    ADXL.frequency(2000000); // 设置 SPI 频率为 2MHz
    chip_sel = 0;
    ADXL.write(0x31); // 选择 DATA_FORMAT 寄存器
    ADXL.write(0x03); // 配置寄存器：4 线 SPI，10 位模式，数据右对齐，量程±16g
    chip_sel = 1;
    chip_sel = 0;
    ADXL.write(0x2D); // 选择 POWER_CTRL 寄存器
    ADXL.write(0x08); // 配置寄存器：进入测量模式
    chip_sel = 1;
}

// 主函数
int main() {
    char buffer[6]; // 存储加速度计数据的缓冲区
    int16_t combine_data[3]; // 存储合成的加速度数据
    float x_axis, y_axis, z_axis; // 存储转换为 g 单位的加速度数据
    initialise(); // 初始化加速度计
    while(1) {
        ThisThread::sleep_for (WATING_TIME); // 等待 0.5 秒
        chip_sel = 0;
        ADXL.write(0xF2); // 发送读取命令，从第一个数据寄存器开始
        for(int i = 0; i < 5; i++) {
            buffer[i] = ADXL.write(0x00); // 读取数据并存储在缓冲区
        }
        chip_sel = 1;

        // 合成 16 位的加速度数据
        combine_data[0] = buffer[1] << 8 | buffer[0];
        combine_data[1] = buffer[3] << 8 | buffer[2];
        combine_data[2] = buffer[5] << 8 | buffer[4];

        // 转换为 g 单位的加速度
        x_axis = 0.016 * combine_data[0];
        y_axis = 0.016 * combine_data[1];
        z_axis = 0.016 * combine_data[2];
printf("x = %+1.2fg\t y = %+1.2fg\t z = %+1.2fg\n\r", x_axis, y_axis, z_axis);
        // 根据 x 轴的加速度控制 LED
        if(x_axis > -0.5 & x_axis < -0.05) {
            lxr1 = 1;
```

```
            lxr2 = lxg1 = lxg2 = 0;
        }
        if(x_axis < -0.5) {
            lxr2 = 1;
            lxr1 = lxg1 = lxg2 = 0;
        }
        if(x_axis > 0.05 & x_axis < 0.5) {
            lxg2 = 1;
            lxr1 = lxg1 = lxr2 = 0;
        }
        if(x_axis > 0.5) {
            lxg1 = 1;
            lxr1 = lxr2 = lxg2 = 0;
        }
        // 根据 y 轴的加速度控制 LED
        if(y_axis > -0.5 & y_axis < -0.05) {
            lyr1 = 1;
            lyr2 = lyg1 = lyg2 = 0;
        }
        if(y_axis < -0.5) {
            lyr2 = 1;
            lyr1 = lyg1 = lyg2 = 0;
        }
        if(y_axis > 0.05 & y_axis < 0.5) {
            lyg2 = 1;
            lyr1 = lyg1 = lyr2 = 0;
        }
        if(y_axis > 0.5) {
            lyg1 = 1;
            lyr1 = lyr2 = lyg2 = 0;
        }
        // 向 PC 发送加速度数据

    }
}
```

Task 5: Further Exploration (Bonus Task worth 10%)

- Modify the code to control the LEDs based on a combination of X and Y-axis readings.

```
#include <mbed.h>
#include "BufferedSerial.h"
#include <cstdio>  // 输入输出头文件

#define WATING_TIME 500ms


#define SCLK A4
#define MISO A5
#define MOSI A6
#define CS D12
#define LXG1 D11
#define LXG2 D9
#define LXR1 D0
#define LXR2 D1
#define LYG1 D6
#define LYG2 A3
#define LYR1 A2
#define LYR2 A0

// 初始化 SPI 和其他外设对象
SPI ADXL(MOSI, MISO, SCLK);
//Serial pc(USBTX, USBRX);
DigitalOut chip_sel(CS);           // 创建数字输出对象用于控制 SPI 通信的片选信号

DigitalOut lxg1(LXG1);
DigitalOut lxg2(LXG2);
DigitalOut lxr1(LXR1);
DigitalOut lxr2(LXR2);
DigitalOut lyg1(LYG1);
DigitalOut lyg2(LYG2);
DigitalOut lyr1(LYR1);
DigitalOut lyr2(LYR2);

// 加速度计初始化函数
void initialise() {
    chip_sel = 1;
    ADXL.format(8, 3); // 设置 SPI 通信格式，8 位数据长度，模式 3
    ADXL.frequency(2000000); // 设置 SPI 频率为 2MHz
    chip_sel = 0;
```

```
    ADXL.write(0x31); // 选择 DATA_FORMAT 寄存器
    ADXL.write(0x03); // 配置寄存器：4 线 SPI，10 位模式，数据右对齐，量程±16g
    chip_sel = 1;
    chip_sel = 0;
    ADXL.write(0x2D); // 选择 POWER_CTRL 寄存器
    ADXL.write(0x08); // 配置寄存器：进入测量模式
    chip_sel = 1;
}

// 主函数
int main() {
    char buffer[6]; // 存储加速度计数据的缓冲区
    int16_t combine_data[3]; // 存储合成的加速度数据
    float x_axis, y_axis, z_axis; // 存储转换为 g 单位的加速度数据
    initialise(); // 初始化加速度计
    while(1) {
        ThisThread::sleep_for (WATING_TIME); // 等待 0.5 秒
        chip_sel = 0;
        ADXL.write(0xF2); // 发送读取命令，从第一个数据寄存器开始
        for(int i = 0; i < 5; i++) {
            buffer[i] = ADXL.write(0x00); // 读取数据并存储在缓冲区
        }
        chip_sel = 1;

        // 合成 16 位的加速度数据
        combine_data[0] = buffer[1] << 8 | buffer[0];
        combine_data[1] = buffer[3] << 8 | buffer[2];
        combine_data[2] = buffer[5] << 8 | buffer[4];

        // 转换为 g 单位的加速度
        x_axis = 0.016 * combine_data[0];
        y_axis = 0.016 * combine_data[1];
        z_axis = 0.016 * combine_data[2];
printf("x = %+1.2fg\t y = %+1.2fg\t z = %+1.2fg\n\r", x_axis, y_axis, z_axis);
        // 根据 x 轴的加速度控制 LED
         if(x_axis > 0.5 && y_axis > 0.5) {
            // 例如：同时向右和向上倾斜
            lxg1 = 1; lyr1 = 1;
            lxr1 = lxr2 = lxg2 = lyr2 = lyg1 = lyg2 = 0;
        } else if(x_axis < -0.5 && y_axis < -0.5) {
            // 例如：同时向左和向下倾斜
            lxr2 = 1; lyr2 = 1;
            lxr1 = lxg1 = lxg2 = lyr1 = lyg1 = lyg2 = 0;
        } else if(x_axis > 0.5 && y_axis < -0.5) {
            // 例如：向右倾斜且向下倾斜
            lxg1 = 1; lyr2 = 1;
            lxr1 = lxr2 = lxg2 = lyr1 = lyg1 = lyg2 = 0;
```

```
        } else if(x_axis < -0.5 && y_axis > 0.5) {
            // 例如: 向左倾斜且向上倾斜
            lxr2 = 1; lyr1 = 1;
            lxr1 = lxg1 = lxg2 = lyr2 = lyg1 = lyg2 = 0;
        } else {
            // 默认情况下关闭所有 LED
            lxr1 = lxr2 = lxg1 = lxg2 = lyr1 = lyr2 = lyg1 = lyg2 = 0;
        }

    }
}
```

Task 6: Reflection

After the completion of programming and testing of Tasks 1 and 2 please reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves **“What you did?”, “What it means?”, and “What next?”**, meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. Please, do not provide a summary of what you did as it will not be accepted.

[Answer in the below space]

What I did?
I integrated an ADXL345 accelerometer with an NUCLEO-L432KC board and developed a program that read acceleration data, controlling LEDs based on the X and Y-axis values. The program initialized the SPI for communication, set up the accelerometer, and mapped the acceleration data to LED states.

What it means?
The project demonstrated real-time data acquisition and actuation based on physical input. The LED responses to acceleration represented a basic form of a feedback system, which is crucial in user interfaces or machine status indicators.

What next?
To enhance the project, I planned to introduce a calibration routine for the accelerometer and expand the LED control logic to include Z-axis data. Additionally, implementing a user interface or connecting to a network for remote monitoring could be the next steps to increase functionality.

Lab Score

Marking Criteria	Student's Score
Task 1 Demonstration (10%)	
Task 2 Demonstration (15%)	
Task 3 Demonstration (15%)	
Task 4 Demonstration (25%)	
Reflection (20%)	
Adherence to coding good practice (15%)	
Bonus Task 5 (10%)	